

INF-07a Ve vztahu k charakteru a velikosti vstupu hodnotím nároky algoritmů; porovnám algoritmy podle různých hledisek a vyberu pro řešený problém ten nejvhodnější.

Na test máš 30 minut. Pracuj samostatně, odpovědi a postup zapisuj čitelně na papír nebo do dokumentu a hotové řešení nahraj do portfolia na Google Disku (fotka, sken nebo soubor). Povolené: kalkulačka; počítač s Pythonem jen k ověření úlohy 4. Kde zadání uvádí počet kroků nebo časy, počítej s nimi; časy jsou orientační.

1. Kolik porovnání nejvýš potřebuje lineární a kolik binární vyhledávání v seřazeném seznamu s 500, 50 000 a 5 000 000 prvky? Které z nich funguje i v neseřazeném seznamu?
2. Algoritmus A potřebuje pro vstup velikosti n přesně $3n$ kroků, algoritmus B $\frac{n^2}{2}$ kroků. Pro která n je B rychlejší? Kolikrát víc kroků potřebuje každý z nich, když vstup zvětšíš 10×? Který zvolíš pro $n = 100\,000$?
3. Program jsi spustil pro různé velké vstupy: $n = 1\,000 \rightarrow 0,02$ s; $2\,000 \rightarrow 0,08$ s; $4\,000 \rightarrow 0,32$ s; $8\,000 \rightarrow 1,28$ s. Jak roste doba výpočtu s n a o jaký typ algoritmu jde? Odhadni dobu pro $n = 32\,000$ a posuď, zda program použiješ pro $n = 1\,000\,000$.
4. Funkce hledá dvojice čísel se součtem 10 a počítá kroky vnitřního cyklu. Co program vypíše? Kolik kroků bude potřeba pro seznam 1 000 čísel a jak počet kroků roste s délkou seznamu?

```
def dvojice(s):
    pocet = 0
    kroky = 0
    for i in range(len(s)):
        for j in range(i + 1, len(s)):
            kroky += 1
            if s[i] + s[j] == 10:
                pocet += 1
    return pocet, kroky

print(dvojice([3, 7, 5, 5, 2, 8]))
```

5. Úloha: zjistit, zda je v seznamu n čísel některé číslo dvakrát. Algoritmus A porovná každé číslo s každým ($\frac{n(n-1)}{2}$ porovnání). Algoritmus B seznam nejdřív seřadí (asi $n \log_2 n$ porovnání) a pak porovná jen sousedy ($n-1$ porovnání). Odhadni počet porovnání obou algoritmů pro $n = 10^6$ a dobu výpočtu při 10^9 porovnání za sekundu. Porovnej je i podle paměti a vyber vhodnější.
6. Čidlo teploty posílá jednu hodnotu každou sekundu po celý den. Program A ukládá všechny hodnoty do seznamu (každá zabere 8 B), program B si pamatuje jen součet a počet hodnot a nakonec vypíše průměr. Kolik paměti na data potřebuje každý program za den? Kdy je nutné použít program A?
7. Vyber vhodný postup a zdůvodni ho podle velikosti a charakteru vstupu: (a) jednou hledáš jméno v neseřazeném seznamu 200 přihlášených; (b) e-shop tisíckrát denně hledá kód zboží v seřazeném katalogu 1 000 000 položek; (c) navigace hledá nejrychlejší trasu v síti s milionem křižovatek – vyzkoušet všechny cesty, nebo použít Dijkstrův algoritmus?; (d) v seznamu 50 neseřazených čísel budeš hledat jen jednu – vyplatí se ho nejdřív seřadit?
8. Simulace 100 000 hodů kostkou trvá 0,06 s. Jak dlouho bude trvat simulace 10 000 000 hodů? Kolikrát přesněji bude odhad pravděpodobnosti, že padne šestka? Vyplatí se to?

Test slouží k doložení výstupu do portfolia. Nehodnotí se body ani známkou – rozhoduje, jestli splníš kritéria na konci dokumentu.

Klíč a řešení

1. **lineární: 500; 50 000; 5 000 000; binární: 9; 16; 23 (nejmenší k , pro které $2^k \geq n + 1$); v neseřazeném seznamu funguje jen lineární vyhledávání.** – ověřuje: počet kroků lineárního a binárního vyhledávání v závislosti na velikosti vstupu
1. Lineární vyhledávání v nejhorším případě porovná všech n prvků.
 2. Binární: $2^9 = 512 \geq 501$, ale $2^8 = 256 < 501 \rightarrow 9$ porovnání.
 3. $2^{16} = 65\,536 \geq 50\,001 \rightarrow 16$; $2^{23} = 8\,388\,608 \geq 5\,000\,001 \rightarrow 23$.
2. **B je rychlejší pro $n < 6$ (pro $n = 6$ oba 18 kroků), pro $n > 6$ je rychlejší A. Při 10× větším vstupu potřebuje A 10× víc kroků, B 100× víc. Pro $n = 100\,000$: A $3 \cdot 10^5$, B $5 \cdot 10^9$ kroků – zvolím A.** – ověřuje: porovnání růstu počtu kroků (lineární × kvadratický)
1. $\frac{n^2}{2} < 3n \Leftrightarrow n < 6$ (pro kladné n).
 2. A: $3 \cdot 10n = 10 \cdot 3n$; B: $\frac{(10n)^2}{2} = 100 \cdot \frac{n^2}{2}$.
 3. $n = 100\,000$: $3n = 300\,000$, $\frac{n^2}{2} = 5 \cdot 10^9$.
3. **Při zdvojnásobení n se čas zčtyřnásobí – kvadratický algoritmus ($t \propto n^2$). Pro 32 000 (4× víc než 8 000): $1,28 \cdot 16 \approx 20,5$ s. Pro 10^6 : $0,02 \cdot 1\,000^2 = 20\,000$ s, asi 5,6 h – nevhodný, je potřeba rychlejší algoritmus.** – ověřuje: odhad nároků algoritmu z naměřených časů
1. Poměry časů: $\frac{0,08}{0,02} = \frac{0,32}{0,08} = \frac{1,28}{0,32} = 4$ při dvojnásobném n .
 2. $t \propto n^2$: 4× větší $n \rightarrow 16$ × delší čas, $1,28 \cdot 16 = 20,48$ s.
 3. 10^6 je 1 000× víc než 1 000: čas $0,02 \cdot 10^6 = 20\,000$ s $\approx 5,6$ h.
4. **(3, 15) – tři dvojice (3 + 7, 5 + 5, 2 + 8) a 15 kroků, tj. $\frac{6 \cdot 5}{2}$. Pro 1 000 čísel $\frac{1\,000 \cdot 999}{2} = 499\,500$ kroků; počet kroků roste kvadraticky, asi $\frac{n^2}{2}$.** – ověřuje: určení počtu kroků algoritmu se dvěma vnořenými cykly
5. **A: asi $5 \cdot 10^{11}$ porovnání, asi 500 s (přes 8 minut); B: asi $2 \cdot 10^7 + 10^6 \approx 2,1 \cdot 10^7$, asi 0,02 s. A nepotřebuje paměť navíc, B musí data seřadit (kopie seznamu, nebo změni pořadí původních dat). Pro velká data zvolím B; pro několik desítek čísel je volba jedno – rozhoduje jednoduchost.** – ověřuje: porovnání dvou algoritmů pro tutéž úlohu podle času a paměti
1. A: $\frac{10^6 \cdot (10^6 - 1)}{2} \approx 5 \cdot 10^{11}$, čas $\frac{5 \cdot 10^{11}}{10^9} = 500$ s.
 2. B: $10^6 \cdot \log_2 10^6 \approx 10^6 \cdot 20 = 2 \cdot 10^7$, plus 10^6 porovnání sousedů.
 3. Čas B: $\frac{2,1 \cdot 10^7}{10^9} \approx 0,02$ s – asi 24 000× rychlejší.
6. **A: $86\,400 \cdot 8 = 691\,200$ B, asi 0,7 MB (v Pythonu kvůli režii seznamu ještě víc) a roste s délkou měření; B: 2 čísla, asi 16 B bez ohledu na délku měření. Program A je nutný, když potřebuješ víc než průměr: medián, graf průběhu, hledání výkyvů, opakované zpracování dat.** – ověřuje: hodnocení paměťových nároků algoritmu podle velikosti vstupu
7. **(a) lineární vyhledávání – řazení by stálo víc než jedno projití 200 prvků; (b) binární vyhledávání – nejvýš 20 porovnání místo až milionu a data už jsou seřazená; (c) Dijkstrův algoritmus – počet všech cest roste exponenciálně, hrubá síla je nepoužitelná; (d) ne – seřazení je dražší než jedno lineární hledání (nejvýš 50 porovnání).** – ověřuje: výběr nejvhodnějšího algoritmu s ohledem na charakter a velikost vstupu
8. **Doba roste přímo úměrně počtu hodů: 100× víc hodů $\rightarrow$ asi 6 s. Typická odchylka odhadu klesá s odmocninou z počtu pokusů: $\sqrt{100} = 10$ × přesnější odhad. Pro ukázkou zákona velkých čísel stačí méně hodů, pro přesný odhad se 6 s vyplatí; další desetinásobné zpřesnění by ale stálo 100× víc času (asi 10 minut).** – ověřuje: vztah mezi počtem opakování, dobou výpočtu a přesností výsledku

Kritéria splnění výstupu

- Správně určí počet porovnání lineárního a binárního vyhledávání a vysvětlí, jak roste s velikostí vstupu (úloha 1).
- Z počtu kroků nebo z naměřených časů rozpozná typ růstu (lineární, kvadratický, logaritmický) a odhadne nároky pro větší vstup – aspoň ve 3 ze 4 úloh 2, 3, 4 a 8.
- Porovná dva algoritmy pro tutéž úlohu podle času i paměti a výběr zdůvodní (úlohy 5 a 6).

- Pro konkrétní situaci vybere vhodný algoritmus s ohledem na velikost a charakter vstupu (seřazenost, počet opakování) – aspoň ve 3 ze 4 případů úlohy 7.

Pro učitele: varianty a přizpůsobení

Obměny: jiné velikosti vstupů a jiné naměřené časy (lineární nebo logaritmický růst), v úloze 5 dvojice řazení výběrem $\times$ vestavěné `sorted()`, v úloze 7 plán projektu (kritická cesta) nebo vyhledávání v telefonním seznamu. Pro silnější žáky: ať časy z úlohy 3 skutečně naměří v Colabu (`time.perf_counter`) pro vlastní program a sestaví tabulku. Pro slabší: připravit tabulku mocnin dvojky a vzorce pro počty kroků. Test lze rozšířit o praktickou část – porovnat lineární a binární vyhledávání měření.

Výstup se probírá v lekcích: 1.+2. H52 Podprogramy (týden 26); 1.+2. H54 Simulace náhody (týden 27); 1.+2. H59 Modelování grafem (týden 30); 1.+2. H60 Efektivita algoritmů (týden 30)